

Del backlog al cambio revisable

Ejecuta una tarea real. Produce evidencia. Abre un PR que otra persona pueda evaluar.

Hoy cerramos un ciclo real



No es el objetivo

Instalar todos los arneses.
Construir un orquestador.
Terminar todo el proyecto.

Sí es el objetivo

Completar una unidad pequeña.
Demostrar qué funciona.
Dejar el siguiente paso claro.

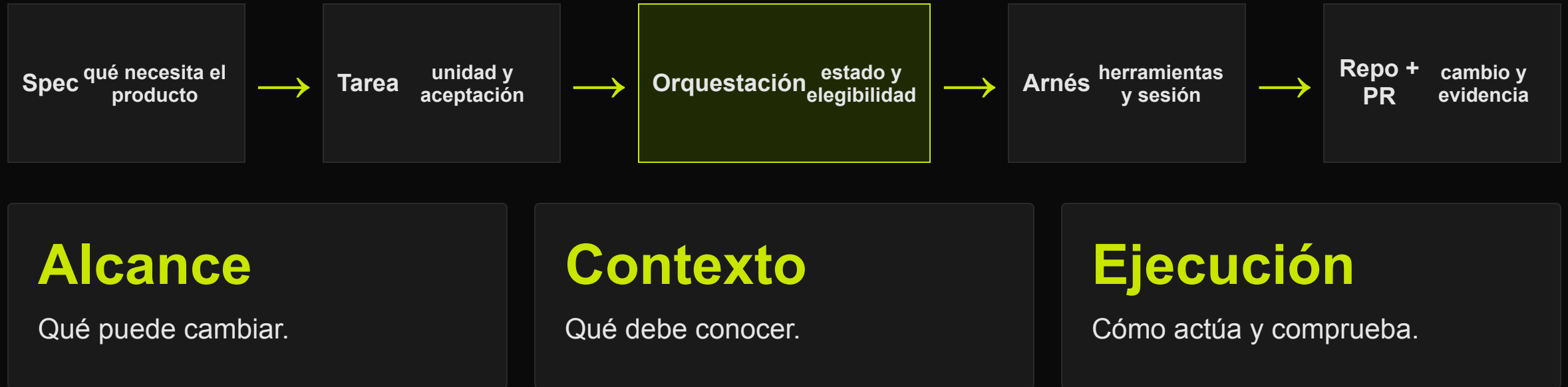
Una ejecución puede terminar sin que la tarea esté resuelta. La evidencia decide.

Hablar poco. Ejecutar mucho.

00–15	Readiness: elegir una tarea sin bloqueos reales
15–30	Dispatch manual guiado: convertir el contrato en un intento
30–70	Implementación individual o en pareja
70–90	Pruebas, evidencia y PR
90–108	Revisión cruzada y corrección
108–120	OpenSymphony, límites de reutilización y cierre

El instructor destraba el proceso; no reemplaza el trabajo del estudiante.

Ubica cada decisión antes de actuar



Spec gobierna intención · tarea gobierna contrato · Linear gobierna estado · GitHub conserva cambio y evidencia.

¿Esta tarea puede empezar ahora?

Lista

- ✓ Dependencias resueltas
- ✓ Alcance y fuera de alcance
- ✓ Aceptación observable
- ✓ Comando de validación real

?

Bloqueada

- ✓ Decisión pendiente
- ✓ Acceso o repo faltante
- ✓ Runner inexistente
- ✓ Contrato ambiguo

Bloqueada no significa fallida. Significa: no inventes; resuelve o elige otra tarea.

Dispatch manual en cuatro campos

Objetivo	Resultado observable de esta tarea, no del proyecto completo.
Límites	Archivos o superficies permitidas; qué queda fuera.
Validación	Pruebas, inspección o comportamiento que demuestra el resultado.
Entrega	Diff, comandos y resultados, riesgos y preguntas pendientes.

“Toma únicamente esta tarea. Si está bloqueada, explica por qué y detente.”

Este contrato funciona con Codex, Claude Code, OpenHands u otro arnés.

Elige la primera tarea ejecutable

Debe caber hoy

- ✓ Un comportamiento o integración
- ✓ Pocos archivos afectados
- ✓ Validación disponible
- ✓ Sin dependencias abiertas

Registra antes de empezar

ID de tarea y issue.
Rama o worktree.
Comando de prueba.
Qué NO vas a cambiar.

Si ninguna tarea está lista, tu trabajo es reparar readiness, no fingir implementación.

Un loop pequeño y verificable

1 · LEER

Issue, tarea, instrucciones y archivos relevantes.

2 · PLANEAR

Plan corto. Riesgo principal. Validación prevista.

3 · CAMBIAR

La modificación mínima que satisface el contrato.

4 · COMPROBAR

Ejecutar pruebas. Inspeccionar diff. Corregir.

Si el agente se desvía, vuelve al contrato. No compenses ampliando el prompt indefinidamente.

Antes de abrir el PR, reúne cuatro pruebas

Diff

Qué cambió y por qué corresponde al alcance.

Validación

Comando ejecutado y resultado real.

Comportamiento

Captura, salida o caso comprobado.

Limitaciones

Qué no se verificó o sigue pendiente.

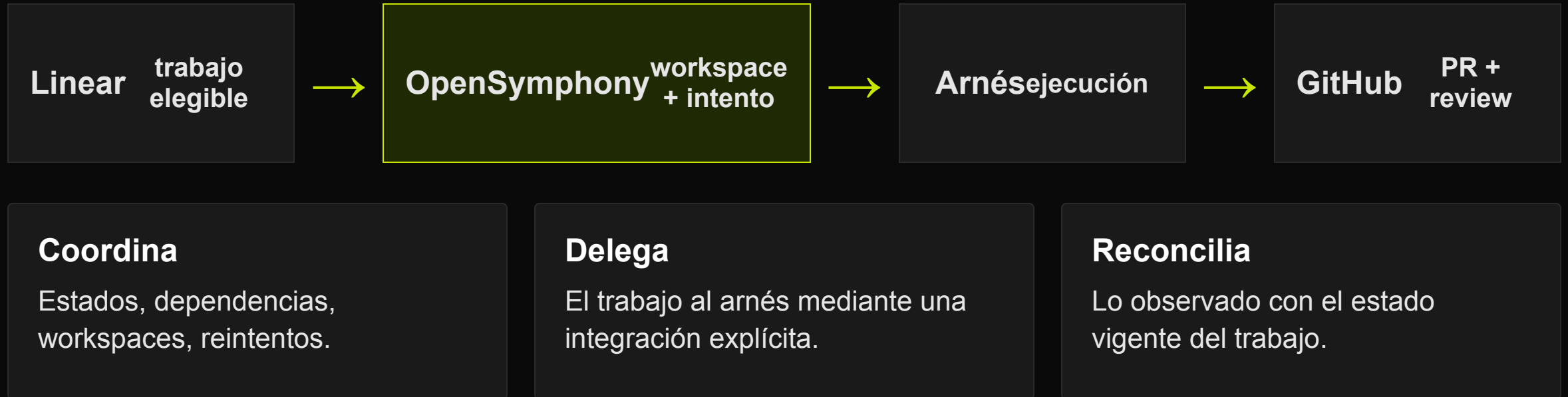
“Ejecutó tests” prueba actividad. “Estos tests pasaron y cubren estos criterios” aporta evidencia.

Review es otra ejecución, no un sello

Pregunta	Evidencia que necesita	Hallazgo útil
¿Cumple el contrato?	Tarea + diff + resultados	Criterio no cubierto
¿Rompe algo?	Tests + callers + invariantes	Regresión concreta
¿Amplió el alcance?	Fuera de alcance + diff	Cambio innecesario
¿Puede integrarse?	CI + conflictos + aprobación	Bloqueo antes del merge

El revisor humano conserva la decisión de merge. La automatización mejora cobertura, no autoridad.

Automatiza coordinación, no criterio mágico



Primero entendimos el ciclo manual. Ahora podemos decidir qué automatizar.

Una institución necesita más que agentes

Hallazgo valioso

El trabajo durable se vuelve un grafo. Aparecen roles, autoridad, memoria, colas, revisión, incidentes y recuperación.

Síntesis aportada por el instructor sobre “The Shape of Things to Come”.

Advertencia operativa

Gas Town fue descontinuado. El propio autor reportó que no logró construir con él de forma confiable. Una arquitectura sugerente no demuestra finalización de tareas.

Comentario citado en AI News; tratar como experiencia reportada, no benchmark.

La metáfora enseña qué responsabilidades emergen. La fiabilidad se demuestra con resultados.

Reutiliza el sustrato. Especializa la política.

1 · Skill + arnés

Una persona, pocas tareas, flujo entendible.

2 · Orquestador reutilizable

Tracker, workspaces, reintentos y review repetibles.

3 · Orquestador propio

Roles, eventos o gobierno inseparables del producto.

OpenSymphony puede generalizar protocolo, ciclo y observabilidad. La política de una organización puede seguir siendo específica.

Construye tu propio orquestador solo cuando los requisitos específicos pagan su costo operativo.

Fallar es un estado; repetir no es una estrategia

Señal	Respuesta mínima	Cuándo escalar
Test falla por el cambio	Corregir dentro del alcance	El contrato es contradictorio
Falta dependencia o acceso	Marcar bloqueada y registrar evidencia	Requiere decisión o permiso externo
Dos intentos sin progreso	Detener, resumir y cambiar estrategia	El mismo bloqueo persiste
Review encuentra regresión	Reabrir la tarea y añadir el caso	Afecta diseño o alcance aprobado

El estado durable debe distinguir completado, fallido, bloqueado y agotado.

Entrega un cambio que otra persona pueda revisar

20 min	Completa o corrige la implementación.	Diff acotado
8 min	Ejecuta validación y reúne evidencia.	Resultados reales
5 min	Abre o actualiza el PR.	Issue enlazado
5 min	Intercambia review con una pareja.	1 hallazgo o aprobación razonada

No hagas merge por terminar el reloj. Cierra solo si aceptación, pruebas y revisión lo permiten.

Deja el sistema mejor preparado para el siguiente intento

Entrega de hoy

- ✓ Issue y tarea alineados
- ✓ PR con evidencia
- ✓ Review registrado
- ✓ Estado siguiente explícito

OpenSymphony
opensymphony.dev

Skills
[OpenSymphony/.agents/skills](https://opensymphony/.agents/skills)

Symphony
github.com/openai/symphony

EBO
[Engineering Behavior](#)
[Observatory](#)

Curso de referencia
Orquestación de trabajo para
ingeniería agencial · Parlina

El objetivo no es más agentes. Es trabajo más trazable, recuperable y verificable.